Modelo_Regresion_IMDb

October 24, 2021

1 Importar Datos

[1]:
```
# Importar librerías
import pandas
import numpy as np
import matplotlib.pyplot as plt
```

[2]:
```
# Lectura de los datos
data = pandas.read_csv("../IMDb_Extensive_Dataset/IMDb movies.csv")
```

```
/opt/conda/lib/python3.9/site-packages/IPython/core/interactiveshell.py:3441:
DtypeWarning: Columns (3) have mixed types.Specify dtype option on import or set
low_memory=False.
  exec(code_obj, self.user_global_ns, self.user_ns)
```

[3]:
```
# Imprimimos información de las variables para saber cuales son numéricas,␣
↪strings, etc.
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 85855 entries, 0 to 85854
Data columns (total 22 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   imdb_title_id         85855 non-null  object
 1   title                 85855 non-null  object
 2   original_title        85855 non-null  object
 3   year                  85855 non-null  object
 4   date_published        85855 non-null  object
 5   genre                 85855 non-null  object
 6   duration              85855 non-null  int64
 7   country               85791 non-null  object
 8   language              85022 non-null  object
 9   director              85768 non-null  object
 10  writer                84283 non-null  object
 11  production_company    81400 non-null  object
 12  actors                85786 non-null  object
 13  description           83740 non-null  object
```

```
 14  avg_vote                85855 non-null  float64
 15  votes                   85855 non-null  int64
 16  budget                  23710 non-null  object
 17  usa_gross_income        15326 non-null  object
 18  worlwide_gross_income   31016 non-null  object
 19  metascore               13305 non-null  float64
 20  reviews_from_users      78258 non-null  float64
 21  reviews_from_critics    74058 non-null  float64
dtypes: float64(4), int64(2), object(16)
memory usage: 14.4+ MB
```

2 Limpieza y nuevas variables

[4]:
```
# Limpieza de Género. Hace dummies para cada género distinto.
generos = []
for j in data['genre'].dropna().str.rsplit(', '):
    for i in j:
        if i not in generos:
            generos.append(i)

generos_names = ["{}{}".format(i,"_Genre") for i in generos]

aux = []
for i in range(len(generos)):
    aux.append(pandas.Series.tolist(data['genre'].str.contains(generos[i],␣
 ↪regex = False)))

Genres = pandas.DataFrame.transpose(pandas.DataFrame(aux, index = generos))

Genres.columns = generos_names
```

[5]:
```
# Limpieza de Lenguaje. Hace dummies para cada lenguaje distinto.
# Faltaría hacer una categoría de otros para reducir la cantidad de variables.
lenguaje = []
for j in data['language'].dropna().str.rsplit(', '):
    for i in j:
        if i not in lenguaje:
            lenguaje.append(i)

lenguaje_names = ["{}{}".format(i,"_Language") for i in lenguaje]

aux = []
for i in range(len(lenguaje)):
    aux.append(pandas.Series.tolist(data['language'].str.contains(lenguaje[i],␣
 ↪regex = False)))
```

```
Lenguajes = pandas.DataFrame.transpose(pandas.DataFrame(aux, index = lenguaje))
Lenguajes.columns = lenguaje_names
```

[6]:
```
# Limpieza de País. Hace dummies para cada pais distinto.
# Faltaría hacer una categoría de otros para reducir la cantidad de variables.
pais = []
for j in data['country'].dropna().str.rsplit(', '):
    for i in j:
        if i not in pais:
            pais.append(i)

pais_names = ["{}{}".format(i,"_Country") for i in pais]

aux = []
for i in range(len(pais)):
    aux.append(pandas.Series.tolist(data['country'].str.contains(pais[i], regex␣
 ↪= False)))

Paises = pandas.DataFrame.transpose(pandas.DataFrame(aux, index = pais))
Paises.columns = pais_names
```

[7]:
```
# Unir los dataframes, para que incluya todas las de generos distintos (los␣
 ↪dummies)
data = pandas.concat([data, Genres], axis=1)
data = pandas.concat([data, Lenguajes], axis=1)
data = pandas.concat([data, Paises], axis=1)
# Quitamos todas las observaciones con NAs
data = pandas.DataFrame.dropna(data)
```

[8]:
```
# Categóricas
# Año
data["year"] = data["year"].astype('category').cat.codes

# Production Company
data["production_company"] = data["production_company"].astype('category').cat.
 ↪codes
data.loc[data['production_company'].isin((data['production_company'].
 ↪value_counts() \
                                          [data['production_company'].
 ↪value_counts() < 35]).index), \
                                          'production_company'] = 0

# Director
data["director"] = data["director"].astype('category').cat.codes
data.loc[data['director'].isin((data['director'].value_counts() \
```

```
                                    [data['director'].value_counts() < 5]).index),␣
↪'director'] = 0

# Writer
data["writer"] = data["writer"].astype('category').cat.codes
data.loc[data['writer'].isin((data['writer'].value_counts() \
                                    [data['writer'].value_counts() < 5]).index),␣
↪'writer'] = 0
```

[9]:
```
data = pandas.get_dummies(data,␣
↪columns=['director',"writer","production_company"])
```

[10]:
```
# USA Gross Income a log de eso limpio
usa_income_series = pandas.Series.dropna(data['usa_gross_income'])
usa_income_series = usa_income_series[usa_income_series.astype('str').str.
↪contains("$", regex = False)]
usa_income_series = pandas.to_numeric(pandas.Series(usa_income_series.
↪astype(str).str.extract('(\d+)')[0]))
data["usa_gross_income"] = np.log(usa_income_series)
```

[11]:
```
# Worldwide Gross Income a log de eso limpio
worldwide_income_series = pandas.Series.dropna(data['worlwide_gross_income'])
worldwide_income_series = worldwide_income_series[worldwide_income_series.
↪astype('str').str.contains("$", regex = False)]
worldwide_income_series = pandas.to_numeric(pandas.
↪Series(worldwide_income_series.astype(str).str.extract('(\d+)')[0]))
data["worlwide_gross_income"] = np.log(worldwide_income_series)
```

[12]:
```
# Budget a log, pero limpio
budget_series = pandas.Series.dropna(data['budget'])
budget_series = budget_series[budget_series.astype('str').str.contains("$",␣
↪regex = False)]
budget_series = pandas.to_numeric(pandas.Series(budget_series.astype(str).str.
↪extract('(\d+)')[0]))
data["budget"] = budget_series
data["budget"] = data["budget"].fillna(0)
```

[13]:
```
data["gross_income"] = data["worlwide_gross_income"] + data["usa_gross_income"]
```

3 Categoricas a dummies

[14]:
```
director_list = [s for s in list(data.columns) if "director" in s]
writer_list = [s for s in list(data.columns) if "writer" in s]
prodcomp_list = [s for s in list(data.columns) if "production" in s]
```

4 Separamos en explicativas y objetivo, y testing y train, y validación

[15]:
```
# Separa la base de datos, usa la variable "quality" como variable objetivo, y
# todas las demás variables como variables explicativas
X = data[["duration", "avg_vote", "votes", \
          "budget","metascore", "reviews_from_users", "reviews_from_critics"] \
         + generos_names + lenguaje_names + pais_names \
         + director_list + writer_list + prodcomp_list].to_numpy()
Y = data["gross_income"].to_numpy()

print(X.shape)
print(Y.shape)
```

```
(6616, 895)
(6616,)
```

[16]:
```
# Separamos en training, validación y testing
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=0.2,␣
 ↪random_state=42)
x_train, x_val, y_train, y_val = train_test_split(x_train, y_train, test_size=0.
 ↪1, random_state=42)

print(f"{len(x_train)} training samples")
print(f"{len(x_val)} validation samples")
print(f"{len(x_test)} test samples")
```

```
4762 training samples
530 validation samples
1324 test samples
```

5 Setup para modelar

[17]:
```
from sklearn.metrics import mean_squared_error
from sklearn.metrics import mean_absolute_error

# Usa kNN de scikit learn
from sklearn.neighbors import KNeighborsRegressor

# Usando Random Forest de scikit learn
from sklearn.tree import DecisionTreeRegressor

# Usando SVR de scikit learn
from sklearn.svm import SVR
```

6 Modelo de KNN

[18]:
```
# Crea un objeto y entrenalo
kNN_regressor = KNeighborsRegressor(n_neighbors=6)
kNN_regressor.fit(x_train, y_train)
```

[18]:
```
KNeighborsRegressor(n_neighbors=6)
```

[19]:
```
# Predict on the validation set
y_val_hat = kNN_regressor.predict(x_val)
print("prediction: ", y_val_hat[:20])
print("true label: ", y_val[:20])
```

```
prediction:  [30.45900506 32.77809525 35.08101898 20.50581334 33.18632706
24.89680102
 32.96840135 34.44298814 34.62782093 33.81639853 33.35614755 29.67324552
 35.12608403 38.47019113 37.32338071 19.91172554 32.78451539 26.4765398
 28.81110086 34.87612148]
true label:  [33.61100799 33.45919768 33.82770811 19.07568788 33.3110332
22.22022779
 33.87879205 36.63961584 33.92198318 35.9230297  36.77521312 22.59535135
 35.10137753 38.35737776 37.87947724 24.18014479 33.97950575 23.15379015
 30.56216544 32.86358718]
```

[20]:
```
# Report accuracy performance on the training and validation set
print(f"Performance on the training set: {kNN_regressor.score(x_train, y_train):
 ↪.3f}")
print(f"Performance on the validation set: {kNN_regressor.score(x_val, y_val):.
 ↪3f}")
```

```
Performance on the training set: 0.735
Performance on the validation set: 0.621
```

[21]:
```
print(mean_squared_error(y_val_hat, y_val))
print(mean_absolute_error(y_val_hat, y_val))
```

```
10.46152111716515
2.355110961013924
```

7 Modelo Decision Tree Regressor

[22]:
```
clf = DecisionTreeRegressor()
clf = clf.fit(x_train, y_train)
```

[23]:
```
# Predict on the validation set
y_val_hat = clf.predict(x_val)
print("prediction: ", y_val_hat[:20])
```

```
print("true label: ", y_val[:20])
```

```
prediction:  [32.44159074 34.95997415 35.09900649 20.19946163 32.30893152
25.08759534
 31.5915386  36.20384545 36.51814337 33.94640121 33.66579918 25.34478274
 36.81048158 39.04742871 37.51059919 28.87737898 33.89060074 20.81106952
 34.0091454  32.58813227]
true label:  [33.61100799 33.45919768 33.82770811 19.07568788 33.3110332
22.22022779
 33.87879205 36.63961584 33.92198318 35.9230297  36.77521312 22.59535135
 35.10137753 38.35737776 37.87947724 24.18014479 33.97950575 23.15379015
 30.56216544 32.86358718]
```

[24]:
```
# Report accuracy performance on the training and validation set
print(f"Performance on the training set: {clf.score(x_train, y_train):.3f}")
print(f"Performance on the validation set: {clf.score(x_val, y_val):.3f}")
```

```
Performance on the training set: 1.000
Performance on the validation set: 0.519
```

[25]:
```
from sklearn.metrics import mean_squared_error
from sklearn.metrics import mean_absolute_error
print(mean_squared_error(y_val_hat, y_val))
print(mean_absolute_error(y_val_hat, y_val))
```

```
13.25129469804425
2.4395608923139145
```

8 Modelo SVR

[26]:
```
svr = SVR()
svr = svr.fit(x_train, y_train)
```

[27]:
```
# Predict on the validation set
y_val_hat = svr.predict(x_val)
print("prediction: ", y_val_hat[:20])
print("true label: ", y_val[:20])
```

```
prediction:  [33.17337358 34.25158311 35.40297399 27.95291134 34.25434583
28.98089343
 34.97421263 33.18715126 36.07273351 35.26487542 32.35290061 33.17085196
 34.86084289 37.79708014 36.65538197 28.05723263 35.6697941  30.70418231
 31.05973937 35.69746799]
true label:  [33.61100799 33.45919768 33.82770811 19.07568788 33.3110332
22.22022779
 33.87879205 36.63961584 33.92198318 35.9230297  36.77521312 22.59535135
 35.10137753 38.35737776 37.87947724 24.18014479 33.97950575 23.15379015
```

```
 30.56216544 32.86358718]
```

[28]:
```
# Report accuracy performance on the training and validation set
print(f"Performance on the training set: {svr.score(x_train, y_train):.3f}")
print(f"Performance on the validation set: {svr.score(x_val, y_val):.3f}")
```

```
Performance on the training set: 0.472
Performance on the validation set: 0.455
```

9 Probando otros modelos y tuning

[29]:
```
# Import models
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import Lasso
from sklearn.neighbors import KNeighborsRegressor
from sklearn.svm import SVR
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error as mse
```

[30]:
```
# Set print options for decimal values
np.set_printoptions(formatter={'float': lambda x: "{0:0.2f}".format(x)})
```

[31]:
```
# Separación en training, validación y testing
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=0.2,␣
 ↪random_state=42)
x_train, x_val, y_train, y_val = train_test_split(x_train, y_train, test_size=0.
 ↪1, random_state=42)

print(f"{len(x_train)} training samples")
print(f"{len(x_val)} validation samples")
print(f"{len(x_test)} test samples")
```

```
4762 training samples
530 validation samples
1324 test samples
```

[32]:
```
# Grid search over kNN regressor
parameters = {'n_neighbors': range(1, 15), 'weights': ['uniform', 'distance'],
              'p': range(1, 2), 'algorithm': ['auto', 'ball_tree']}

gs_model = GridSearchCV(KNeighborsRegressor(), parameters, n_jobs=-1)
gs_model.fit(x_train, y_train)
```

```
print(f"Best score: {gs_model.best_score_:.3f}")
print(f"Best params: {gs_model.best_params_}")
```

```
Best score: 0.636
Best params: {'algorithm': 'ball_tree', 'n_neighbors': 12, 'p': 1, 'weights':
'uniform'}
```

[33]:
```
# Predict on validation data, and print score
y_val_hat = gs_model.best_estimator_.predict(x_val)

print("prediction: ", y_val_hat[:10])
print("true label: ", y_val[:10].astype('float'))

print(f"MSE on the training set: {mse(gs_model.best_estimator_.
 ↪predict(x_train), y_train):.3f}")
print(f"MSE on the validation set: {mse(gs_model.best_estimator_.
 ↪predict(x_val), y_val):.3f}")
print(f"MSE on the test set: {mse(gs_model.best_estimator_.predict(x_test),␣
 ↪y_test):.3f}")
```

```
prediction:  [30.86 33.83 35.73 19.74 33.81 25.14 32.66 35.37 34.22 34.86]
true label:  [33.61 33.46 33.83 19.08 33.31 22.22 33.88 36.64 33.92 35.92]
MSE on the training set: 8.171
MSE on the validation set: 10.415
MSE on the test set: 11.058
```

[34]:
```
# Grid search over Lasso Regularized Linear Regression
parameters = {'normalize': ['True', 'False'], 'alpha':np.logspace(-10, 0,base =␣
 ↪np.e, num=100, endpoint = True).tolist()}

gs_model = GridSearchCV(Lasso(alpha = 0.1), parameters, n_jobs=-1, scoring='r2')
gs_model.fit(x_train, y_train)

print(f"Best score: {gs_model.best_score_:.3f}")
print(f"Best params: {gs_model.best_params_}")
```

```
Best score: 0.554
Best params: {'alpha': 0.0005672289891977495, 'normalize': 'True'}
```

[35]:
```
# Predict on validation data, and print score
y_val_hat = gs_model.best_estimator_.predict(x_val)

print("prediction: ", y_val_hat[:10])
print("true label: ", y_val[:10].astype('float'))
```

```
print(f"MSE on the training set: {mse(gs_model.best_estimator_.
 ↪predict(x_train), y_train):.3f}")
print(f"MSE on the validation set: {mse(gs_model.best_estimator_.
 ↪predict(x_val), y_val):.3f}")
print(f"MSE on the test set: {mse(gs_model.best_estimator_.predict(x_test),␣
 ↪y_test):.3f}")
```

```
prediction:  [32.13 35.03 34.34 27.59 33.26 30.43 34.52 32.75 34.19 33.56]
true label:  [33.61 33.46 33.83 19.08 33.31 22.22 33.88 36.64 33.92 35.92]
MSE on the training set: 10.785
MSE on the validation set: 12.426
MSE on the test set: 12.904
```

[ ]:
```
# Grid search over SVR
parameters = {'kernel': ['linear', 'poly', 'rbf'], 'degree': range(1, 4),
              'coef0': [0.0, 0.001, 0.01, 0.1, 1.0], 'C': [0.001, 0.01, 0.1, 1.
 ↪0, 10, 100]}

gs_model = GridSearchCV(SVR(), parameters, cv=2, n_jobs=-1, verbose=2)
gs_model.fit(x_train, y_train)

print(f"Best score: {gs_model.best_score_:.3f}")
print(f"Best params: {gs_model.best_params_}")
```

[ ]:
```
# Predict on validation data, and print score
y_val_hat = gs_model.best_estimator_.predict(x_val)

print("prediction: ", y_val_hat[:10])
print("true label: ", y_val[:10].astype('float'))

print(f"MSE on the training set: {mse(gs_model.best_estimator_.
 ↪predict(x_train), y_train):.3f}")
print(f"MSE on the validation set: {mse(gs_model.best_estimator_.
 ↪predict(x_val), y_val):.3f}")
print(f"MSE on the test set: {mse(gs_model.best_estimator_.predict(x_test),␣
 ↪y_test):.3f}")
```